

Documento técnico de código fuente

Este documento explica cómo está organizado y cómo funciona el código del proyecto `prueba-AI`.

Resumen

- Proyecto: asistente local RAG para academia de idiomas.
- Carpetas principales: `backend/`, `frontend/`, `db/`, `docs/`.

Estructura del proyecto

```
prueba-AI/
  README.md
  backend/
    analizar/                # Documentos fuente (.md/.txt)
    requirements.txt
    main.py                  # API y orquestación
    rag_engine.py            # Indexación y embeddings
    ollama_client.py         # Cliente para Ollama
    skills.py                # Habilidades deterministas (inscripciones)
  db/
    schema.sql              # Script SQL de ejemplo
  frontend/
    package.json
    src/                     # Código Vue 3
  docs/
    design_document.md
    erd.md
```

Descripción de carpetas y módulos

- `backend/main.py`:
 - Punto de entrada de la API FastAPI.
 - Registra inicialización del RAG en segundo plano y expone `/health` y `/chat`.
 - Intercepta solicitudes de inscripción y rutas deterministas antes de llamar al flujo RAG.
- `backend/rag_engine.py`:
 - Carga archivos (`backend/analizar/`) y los fragmenta en “chunks”.
 - Solicita embeddings a Ollama mediante `POST /api/embeddings` y guarda embeddings en memoria.
 - `retrieve(query, top_k)` devuelve los chunks más relevantes por similitud coseno.
- `backend/ollama_client.py`:
 - Construye el prompt fundamentado con el contexto recuperado y llama a `OLLAMA_BASE_URL/api/generate`.
 - Contiene el sistema prompt que obliga a responder sólo desde contexto y en el idioma requerido.
- `backend/skills.py`:
 - Funciones deterministas: extracción de campos de inscripción (`extract_registration_details`), validaciones, y persistencia atómica en `student_registrations.json`.
 - Implementa buenas prácticas: validación, bloqueo (`REGISTRATION_LOCK`) y escritura atómica con archivo temporal.
- `frontend/`:
 - Interfaz Vue 3 y Vite. `ChatInterface.vue` gestiona la sesión y envía `session_id` para permitir borradores de inscripción.

Fragmentos relevantes

- Manejo de salud y RAG en `main.py` (simplificado):

```
@app.get("/health")
def health_check():
    return {"api": "ok", "ragReady": rag_ready, "ragError": None if rag_ready else rag_error}

• Escritura atómica en skills.py:

with os.fdopen(fd, 'w', encoding='utf-8') as f:
    json.dump(registrations, f, indent=2, ensure_ascii=False)
os.replace(temp_path, REGISTRATIONS_FILE)
```

Requisitos y dependencias

- Backend: requirements.txt incluye fastapi, uvicorn[standard], langdetect, numpy, requests, python-dotenv.
- Frontend: package.json (Vue 3, Vite, Tailwind). Para Tailwind ejecutar `npm install tailwindcss @tailwindcss/vite`.
- Ollama: Servicio local accesible en OLLAMA_BASE_URL (default `http://127.0.0.1:11434`).

Flujo de una consulta (alto nivel)

1. Cliente envía POST `/api/chat` con prompt (y opcional `session_id`).
2. `main.py` detecta idioma y comprueba si la intención es una inscripción; aplica habilidades deterministas.
3. Si no es una skill y el índice RAG está listo, `rag_engine._get_embedding(query)` obtiene embedding de la consulta.
4. `rag_engine.retrieve()` devuelve los chunks más relevantes.
5. `ollama_client.generate_response()` construye el prompt con contexto y llama a Ollama.
6. Respuesta devuelta al cliente; además se guarda en cache semántica.

Buenas prácticas aplicadas

- Evitar exposición de datos personales al LLM (inscripciones validadas de forma determinista).
- Escritura atómica para evitar corrupciones de `student_registrations.json`.
- Separación de responsabilidades: indexación, cliente Ollama, skills y API.

Cómo extender o migrar

- Migrar `student_registrations.json` a una base de datos relacional usando `db/schema.sql`.
- Externalizar el almacenamiento de embeddings a un vector DB si el conjunto de documentos crece.
- Añadir pruebas unitarias y de integración para endpoints críticos.

Dónde revisar el código

- `backend/main.py` — comportamiento general del API.
- `backend/rag_engine.py` — carga y recuperación de documentos.
- `backend/ollama_client.py` — generación de prompts y llamadas a Ollama.
- `backend/skills.py` — reglas de negocio y persistencia.

Si quieres, puedo generar un PDF con este documento o añadir fragmentos de código adicionales comentados para la entrega.